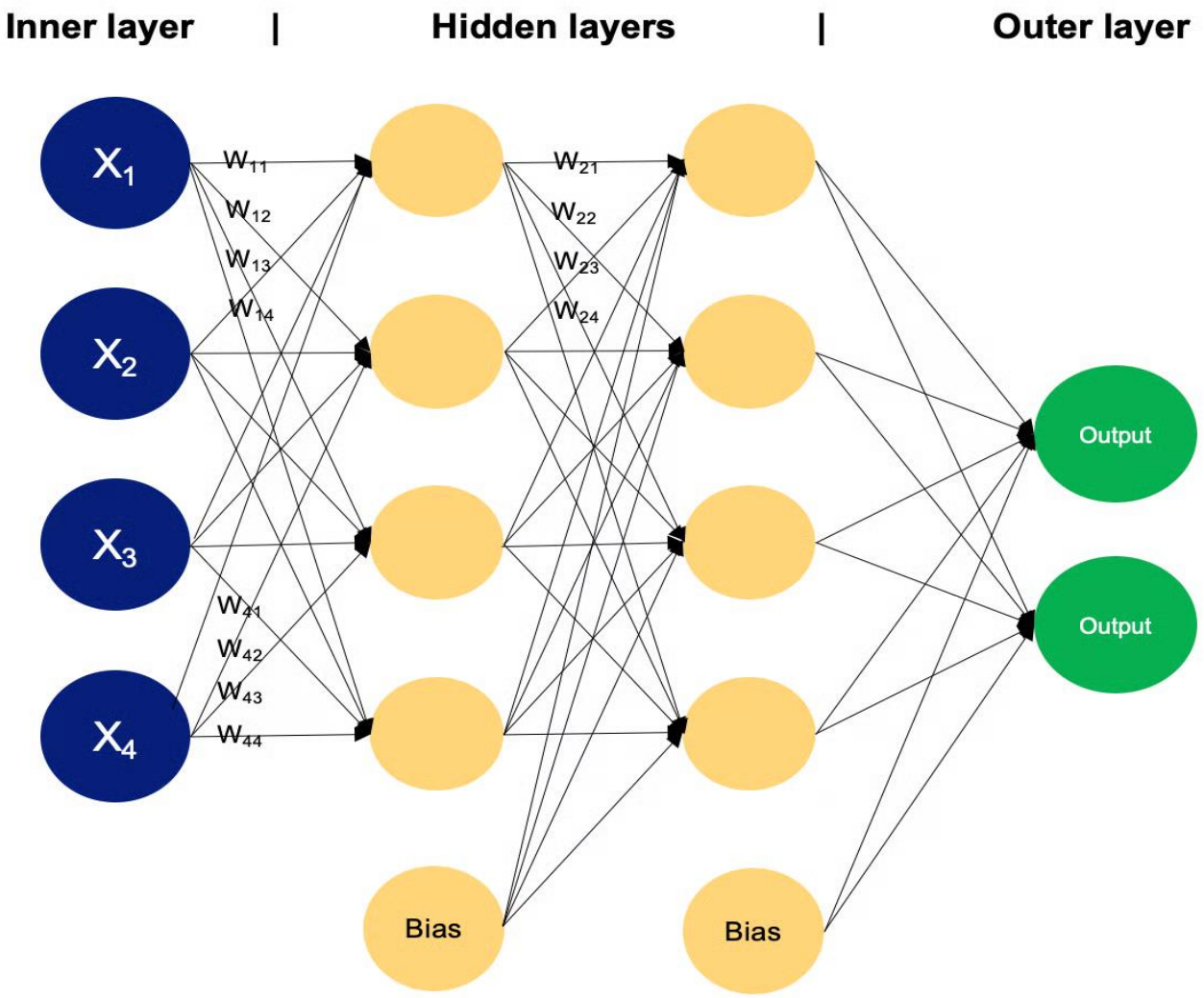
Creating Your First Neural Network in Python with Tensorflow



Creating Your First Neural Network in Python with Tensorflow

steps:

1. Define our training data
2. Create our model
3. Add layers
4. Compile our model
5. Fit our model to our training data
6. Predict values

```
pip install tensorflow --upgrade
```

- Layer type: The type of layer you are creating (ie: Dense, Bidirectional)

- Activation function: A function that modifies the input of a neuron(ie: relu, sigmoid)

- Loss function: A function that measures the inaccuracy of a model in correctly predicting the training data(ie: mean squared error, categorical cross entropy)

- Optimization function: The algorithm that is minimizing the loss value of the model. Is what is actually doing the training(ie: Adaptive Moment Estimation, stochastic gradient descent)

•Loss function: A function that measures the inaccuracy of a model in correctly predicting the training data(ie: mean squared error, categorical cross entropy)

Mean Squared Error (MSE) : One of the most popular loss functions, MSE finds the average of the squared differences between the target and the predicted outputs

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$$

•Optimization function (Optimizers): The algorithm that is minimizing the loss value of the model. Is what is actually doing the training(ie: adam(Adaptive Moment Estimation), sgd (stochastic gradient descent), Adagrad)

Optimizers are algorithms or methods used to change the attributes of the neural network such as weights to reduce the loss

- Batch Gradient descent
- Stochastic Gradient Descent
- Minibatch Stochastic Gradient Descent
- Adam
- RMSProp
- AdaDelta
- Adagrad

Gradient descent:

When we carry out forward and backpropagation using gradient descent, we can use basically one of three methods:

1. Stochastic gradient descent
2. Mini-batch gradient descent
3. Batch gradient descent

Training Dataset

Subject	Features (Inputs)			Label (Target)
	X1	X2	X3	Y
(1) 1	20	60	44	1
2	15	48	69	0
3	02	59	82	1
4	40	45	74	0

1.If we input **one training example at a time** and carry out forward and backpropagation to update the weights, that is called **Stochastic gradient descent.**

The number of passes = 4 passes (Forward pass and Backward pass) = 1 Epoch.

Training Dataset

(2)

Subject	Features (Inputs)			Label (Target)
	X1	X2	X3	Y
1	20	60	44	1
2	15	48	69	0
3	02	59	82	1
4	40	45	74	0

2. If we input **Multiple training example at a time** (less than the entire training set) that is called **Mini-batch gradient descent**.

The number of passes = 2 passes (Forward pass and Backward pass) = 1 Epoch.

Training Dataset

Subject	Features (Inputs)			Label (Target)
	X1	X2	X3	Y
1	20	60	44	1
2	15	48	69	0
3	02	59	82	1
4	40	45	74	0

(3)

3. If we input **the entire training dataset** and carry out forward and backpropagation, that is called **Batch gradient descent.**

The number of passes = 1 pass (Forward pass and Backward pass) = 1 Epoch.

In Batch gradient descent, we have to process the entire dataset to update the weights, which can be **computationally very expensive** if the dataset is **large**

So, Mini-batch or Stochastic gradient descent is faster than batch gradient descent.

- Most Deep learning algorithms are based on an Optimization algorithm called stochastic gradient descent. It is generally the preferred method for basic Backpropagation.

Creating Your First Neural Network in Python with Tensorflow

```
pip install tensorflow --upgrade
```

steps:

1. Define our training data
2. Create our model
3. Add layers
4. Compile our model
5. Fit our model to our training data
6. Predict values

Creating and Testing Our Model

Dataset

X1 (Input)	X2 (Input)	Y (Output)
0	0	0
0	1	1
1	0	1
1	1	0

Import the TensorFlow and Keras libraries.

```
Import tensorflow as tf
From tensorflow import keras
From tensorflow.keras import layers
```

Define our training input and output data with type 16 bit float
Each input maps to an output

```
X = tf.constant([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=tf.float16)
Y = tf.constant([[0], [1], [1], [0]], dtype=tf.float16)
```

Create a new Sequential Model

```
model = keras.Sequential ()
```

Add our layers

```
model.add(layers.Dense( 4,        input_dim=2,        activation='relu'))
```

Amount of Neurons .كمية الخلايا العصبية

Define an input dimension because this is the first layer

Use relu activation function

```
model.add(layers.Dense(1,        activation='sigmoid' ))
```

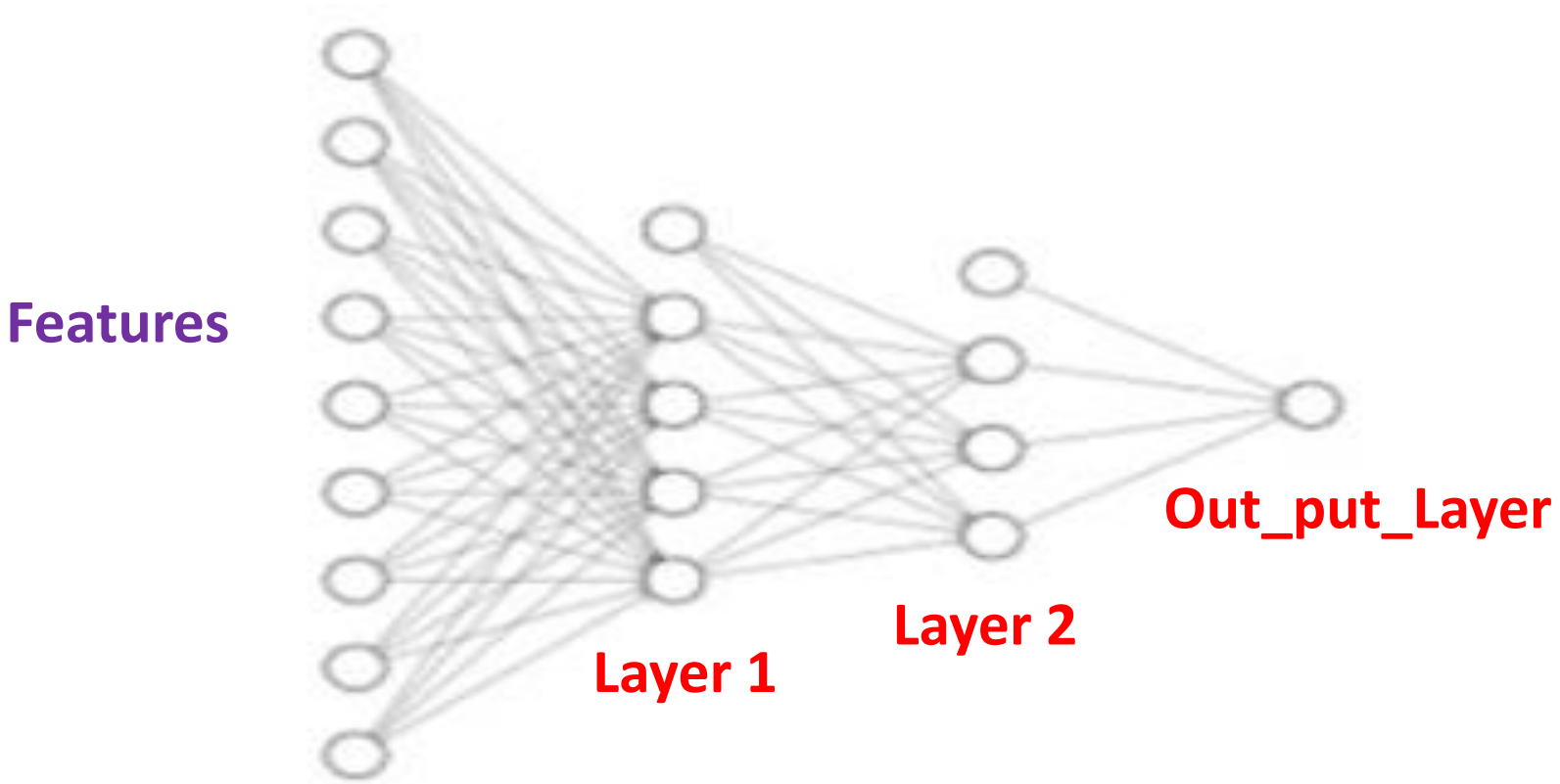
Amount of Neurons. We want one output

Use sigmoid because we want to output a binary classification

Softmax in multiclass

Example of model with 8 features, 4 neurons in layer1, 3 neurons in layer 2 and 1 neuron in output layer

```
model = keras.Sequential ()
model.add(layers.Dense(4, input_dim=8, activation='relu', name='layer 1'))
model.add(layers.Dense(3, activation='relu' , name='layer 2'))
model.add(layers.Dense(1, activation='sigmoid' , name='Output_layer'))
```



Compile our layers into a model

```
model.compile(loss='mean_squared_error', optimizer='adam', metrics=['binary_accuracy'] )
```

The loss function that is being minimized

Our optimization function

Metrics are different values that you want the model to track while training

Fit our model to our training data

```
model.fit(X,        Y,        epochs=200,        verbose=1 )
```

Input training data

Output training data

Amount of iterations we want to train for

Amount of detail you want shown in terminal while training